Towards Parallelising Smart Contracts Execution in Go-Ethereum: An Analysis Report

Roy Friedman
Computer Science
Technion

Abstract

In this interim report, we analyze the possibility of parallelizing the geth virtual machine implementation. We identify the exact place in the code where parallelization should be introduced, design a sketch of how it should be realized, and also identify the parts of the system that should be protected to enable safe multi-threaded operation.

1 Introduction

Go-Ethereum, also known as geth, is a widely used Golang execution layer implementation of the Ethereum protocol [6]. The current implementation of geth is sequential, that is, transactions are executed one after the other. This is unfortunate, since modern CPUs are multi-threaded, and there are many indications that performance could be improved by resorting to parallelism [2, 4, 11, 12]. To that end, we analyze the code of geth, as found in [6]. We identify the exact location in which the parallelization of smart contract execution should be introduced. We then sketch the solution and explain how it should be realized. We further identify which parts and objects in the system are vulnerable to parallelism, and discuss how to protect them with minimal overhead.

We note that there may be other aspects of the system that could enjoy thread level or SIMD parallelism. Yet, here we focus on parallelizing the execution of smart contracts under the assumption that this where the main performance bottleneck is.

Disclaimer: The analysis here refers to the Sprouted Seed Vial (v1.16.2) version of geth found in [6]. Also, it is possible that as we start implementing we will discover additional data structures that are affected by introducing parallelism and were missed by the current analysis.

2 Preliminaries

We refer to smart contracts in their typical Ethereum context and semantics as explained in [7]. An Ethereum block consists of a sequence of transactions, each can either be a simple transfer transaction or a smart contract invocation. Each transaction may access various objects for both reading and writing. The set of objects accessed by a given transaction for reading is known as its readset, while the set of transactions access for writing is called its writeset. Whenever two transactions tx_1 and tx_2 access the same object obj and at least one of these accesses is a write, we say that tx_1 and tx_2 are in conflict; this fact is denoted $tx_1 \bowtie tx_2$. If obj is in the writesets of both tx_1 and tx_2, we identify this as a write-write conflict, denoted WW conflict. If obj is in the readset of one of these two transactions and in the writeset of the other, we say refer to this as a read-write conflict, denoted RW-conflict.

Finally, given a set of transactions $\mathcal{T}$, such that all the transactions are of the same block, we can represent these transactions and their conflicts as a conflict graph $G = (V, E)$ such that $V = \mathcal{T}$ and

```
92          // Iterate over and process the individual transactions
93          for i, tx := range block.Transactions() {
94              msg, err := TransactionToMessage(tx, signer, header.BaseFee)
95              if err != nil {
96                  return nil, fmt.Errorf("could not apply tx %d [%v]: %w", i, tx.Hash().Hex(), err)
97              }
98              statedb.SetTxContext(tx.Hash(), i)
99
100             receipt, err := ApplyTransactionWithEVM(msg, gp, statedb, blockNumber, blockHash, context.Time, tx, usedGas, evm)
101             if err != nil {
102                 return nil, fmt.Errorf("could not apply tx %d [%v]: %w", i, tx.Hash().Hex(), err)
103             }
104             receipts = append(receipts, receipt)
105             allLogs = append(allLogs, receipt.Logs...)
106         }
```

Figure 1: The main execution loop of geth

$E = \{(i,j)|tx_i \bowtie tx_j\}$. That is, the set of vertices is exactly the same of transaction $\mathcal{T}$, and there is an edge in the graph between any pair of transactions tx_i and tx_j if they conflict (any type of conflict).

3 The Main Execution Loop of geth

The main execution loop is in the file `core/state_processor.go`, inside the `process(...)` function, line 93, where an iterator iterates over all the blocks transactions - `block.Transactions()` - is invoked. See listing in Figure 1. Each transaction is then handled by the function `ApplyTransactionWithEVM(...)` in this same file. Here, the transaction goes through a set of wrapper functions until it is finally executed, and then the transaction is finalized by `ApplyTransactionWithEVM(...)` with the update of the DB and Merkle tree.

The first of these wrappers is `ApplyMessage(...)`, defined in `internal/ethapi/api.go`, which invokes `applyMessageWithEVM(...)`, which then calls the core.ApplyMessage(...) function, defined inside the file `core/state_transition.go`. Essentially, `applyMessageWithEVM(...)` wraps the execution of of the function `core.ApplyMessage(...)` with timeout and cancellation logic, e.g., to ensure bounded execution time. Similarly, `core.ApplyMessage(...)` is another wrapper to the actual execution invoked inside the `execute(...)` function, which is defined in `core/state_transition.go`. As part of the latter's code, if the transaction is a new contract, it is executed by `st.evm.Create(...)`, or otherwise by `st.evm.Call(...)`. Ultimately, both call `evm.interpreter.Run(...)` where the transaction is finally executed. Briefly, the above wrapper functions handle various aspects of gas calculations, both per transaction and per-block, interpret the smart contract's code, warm up declared objects, and prepare the environment for the transaction execution and perform post-execution cleanups.

4 The Road to Parallelism

The first step towards concurrency is to replace the iterator in line 93 of `core/state_processor.go`, with the following logic:

1. Generating a conflict graph.
2. Color the graph with a heuristic deterministic minimal coloring algorithm such as [3].
3. Direct edges (between conflicting transactions) according to their respective colors' numbers.
4. Loop as long as there are transactions in the graph and do:
   (a) Fetch a transaction with no incoming edges.

(b) Schedule this transaction to run in a concurrent go-routine, including finalization and cleanup logic.

(c) Remove the transaction and all its outgoing edges from the conflict graph.

Notice that one of the benefits of using the graph is that we do not need to protect the objects store, unless we wish to implement the write-write optimization of [1]. Also, we note that transacrtions by the same client should be ordered according to their nonce order.

The main things to worry about are the shared state objects such as tries, Merkle trees, etc. For example, the evm's context might be an issue. Other places to worry about inside `execute()` are `st.state.Prepare()` and `st.state.AddAddressToAccessList(addr)`. See detailed analysis in Section 5 below.

Moving Forward Plan

1. A prototype that assumes the readsets and writesets are fully and accurately declared in accordance with EIP-2930 [8], and enforce all conflicts (RW+WW).

   (a) For correctness, we would start by synchronizing all access to shared data.

   (b) Once we have a working prototype, we would start reducing the amount the synchronization based on the analysis in Section 5 below and other insights discovered along the way.

2. Improve (1) by avoiding enforcement of WW conflicts as suggested in [1]. This would require replacing StateDB with a multi-versioned instance and making sure that writes are assigned a version number that is consistent with the color number of the TX and the TX "rank" within its color. As we discuss below, since all mutations to the same object are accuulated and update the database only once the transaction commits, and since reads are performed from a snapshot of the DB at the beginning of the execution, this should be relatively simple to implement.

3. Improving on both (1) and (2) by removing the assumption about complete and accurate declaration of readsets and writesets. This will probably not be accomplished within the funded period.

5 Making the Code Thread Safe

StateDB (`core/state/state_object.go`) This data structure holds account balances, information required for snapshots, journal, logs, and debug related data. It is mutated by every transaction.

The actual state of objects read and written to by transactions is held in `StateDB.db`. Using coloring, it is tempting to think that accesses to `StateDB.db` should be safe, since we ensure that there are no conflicting accesses to the same objects using the color based scheduling. In practice, however, the internal implementation of `StateDB.db` maintains a trie and a journal and its operation is more intricate. Simply turning all accesses to synchronized (atomic) would ensure correctness, but is likely to be expensive. Due to the centrality of handling StateDB efficiently and correctly, we expand its analysis in Section 5.1 below.

GasPool (`core/gas_pool.go`) This object tracks block-wide remaining gas. It should be protected since it is shared and updated by all transactions in a block. We can use a per-thread gas accounting or pre-partition gas limits in order to reduce the number of synchronized (atomic) access to the actual shared object. All accesses to GasPool are already performed through accessor functions, which simplifies the task.

EVM and EVMInterpreter (`core/vm/`) This is the execution engine with per-call context. Since EVMInterpreter is instantiated one per EVM, it is safe to run multiple EVMs concurrently. Section 5.2 below explores the feasibility of instantiating a fresh EVM (and EVMInterpreter) per transaction or per thread.

Logs and Receipts Each transaction appends logs and receipts to global arrays, which obviously need to be synchronized. To reduce the cost of invoking a synchronized access for each individual transaction, we may use local slices per worker, then merge after parallel execution.

Block Context / ChainConfig these data structures include base fee, timestamp, number, etc. However, they are read-only, so they are already thread-safe.

Precompiled Contracts Some precompiles, e.g. BLS, might use global caches or native code. Hence, we must ensure any native code called is thread-safe. We should guard any global caches, e.g., through mutex in precompile.

5.1 A Deeper Analysis of StateDB

StateDB consists of the following fields:

db: Database Database is an interface defined in `core/state/database.go` that abstracts state access, whose default implementation is `CachingDB`, which is not entirely not thread (go-routine) safe.

Specifically, committing updates to the `trie` are thread safe and already internally guarded by a mutex, and similarly snapshot creations are also thread safe. The other high level methods should be made thread safe using a combination of exclusive locks for writing + shared locks for reading. We may also consider replacing mode of the exclusive locks with a copy-on-write approach. Practically, we recommend starting with high-level locking to ensure correctness. Then. whenever we will discover contention, to replace these locks with more fine grain locking, and potentially lock-free solutions.

prefetcher: this is geth's background loader that preloads account/storage trie nodes likely to be needed soon, so when hashing/committing the state (or reading code/slots) the data is already in memory. `triePrefetcher` hides disk latency by overlapping trie IO with execution and hashing, improving block processing throughput and providing useful metrics about prefetch efficiency. Need to make its calls thread safe, and ensure it does not gets deleted in the middle of a block's execution.

reader: this is the read-only view of state pinned to a specific root. It lets StateDB (and its `stateObjects`) fetch data from the underlying tries/DB without mutating anything. That is, `reader` is the snapshotted, cached, read-only state access layer that StateDB uses to load accounts, storage, and code from the database for the current root, while mutations happen elsewhere. This field does not get updated after being assigned.

stateObjects: a hashmap in-memory cache of live account objects being read/modified during block/tx execution. It accumulates changes across transactions, and is being updated at the end of each transaction inside the `Finalize()` function. When the entire block is committed, it is used to update the trie etc., but it is not reset. Only deleted objects are removed from `stateObjects` and thus it can potentially grow in an unbounded manner. Hence, accesses to this field should be synchronized.

stateObjectsDestruct: a hashmap of all in-memory account objects being deleted. This field accumulates deleted objects from all transactions. It is initialized for each block. It is populated at the end of each transaction inside the `Finalise(...)` function. Hence, for concurrency, it should be made thread safe. It is tempting, for efficiency, to have the entire loop should be synchronized once rather than invoking synchronization for each access. However, it is also accessed inside `SetStorage(...)` and it is also copied when the state is cloned.

mutations: is a hashmap of per-account "pending operation" index that summarizes what should happen to each touched account when the block's changes are materialized. It coalesces many transactions edits into a single update per address so the commit phase can do minimal. It is cleared after each transaction terminates. Hence, for concurrency, each transaction or thread should have its own `mutations` object.

logs: the per-transaction logs emitted by contracts (used in GetLogs). The logs data structure is a hashmap whose key is the transactions hash and the value is the log for that transaction. Obviously, one should use a concurrent hashmap – the default in Go is `ConcurrentMap`, so this should be our starting point.

journal: is the undo log for all in-memory state changes during EVM execution. Its job is to let Geth snapshot the current state (`Snapshot()`), apply a bunch of mutations while running a tx or call, and revert those mutations if the transaction/call reverts or an error occurs (`RevertToSnapshot(id)`), and ultimately, finalize/clear at the right boundaries. The journal is clreaed at the finalization of each transaction. Hence, for concurrency, each transaction or thread should have its own journal.

accessList: EIP-2929/EIP-2930 access tracking for gas calculation and Verkle witness. There is a single copy held by StateDB. It is seeded at tx start via `StateDB.Prepare(...)`, and then expanded lazily during opcode execution on first cold touches (addresses or slots) in `core/vm/operations_acl.go`. This is how EIP-2929/2930 "warm vs. cold" costs are enforced and tracked. All other accesses are read only. For safe concurrent execution, we need to maintain a copy per transaction, or per thread, rather than a single copy.

accessEvents: is the Verkle/stateless tracker that records which addresses / storage slots / code chunks were touched (read/write) during execution. It seems that the tracking during a transaction's execution is performed inside the EVM, and is merged into the overall field at the end of the transaction's execution, in `core/state_processor.go`. Hence, naïve synchronization of the merge at the end of the transaction should be fine here.

preimages: this is a hashmap that stores SHA3 preimages for opcode SHA3 debugging/tracing. We should replace hashmap with Go's concurrent hashmap. Maybe we can make private per transaction or thread and merge at the end of the block?

originalRoot / trie: connects the in-memory state to the underlying Merkle Patricia Trie. originalTree gets updated inside the `commit()` function, called after a transactions finishes its execution correctly. This part should be protected.

witness: used for stateless execution and proof generation. At the beginning of each transaction execution, it is attached to `stateless.witness` and detached from it when the transaction terminates. `stateless.Witness` is the structure geth uses to collect a state witness (accounts, storage slots, and code chunks actually touched, being updated on each corresponding read) so block processing can be verified statelessly—especially in Verkle mode. For concurrent execution, we need to maintain the witness on a per transaction or per-thread basis rather than one per the entire single statedb.

Various metrics fields: — execution time and count measurements for profiling. Here we need to ensure that updates are atomic, and performed once per transaction, and if possible per thread at the end of a block's execution, in order to reduce synchronization overhead.

5.2 A Deeper Analysis of EVM and EVMInterpreter

It seems that indeed EVM and EVMInterpreter do not have any persistent/lingering state between invocations of transactions inside them. It is possible to move `vm.NewEVM(...)` inside the tx loop and pass that new EVM into `ApplyTransactionWithEVM(...)`. It will work because (a) the block context is the same for all txs in the block, and (b) the StateDB (with its journal/snapshots) already provides transaction-level isolation and rollback. For better performance, it is preferable to have a separate VM per thread.

As for EVMInterpreter, the current impelementation in any case allocates a fresh EVMInterpreter for the new VM. The main thing to worry about seems to be caching of interpreted smart contracts and debug related information, where sharing can be useful for performance.

6 Related Projects

Erigon [5], also known as Turbo-Geth, is an alternative implementation to geth that emphasizes space efficiency, modularity, and performance. In particular, it supports parallel execution of transactions. However, as it has modified much of the internals, it is not very useful for us.

Nevermind [10] in an EVM client built on the ,NET framework. It supports executing transactions in parallel through speculative execution. This can be helpful and insightful when we reach step (3) in the moving forward plan of Section 4.

Neon [9] enables deploying EVM dApps on Solana, thereby benefitting from Solana's capability to run transactions in parallel. Hence, it is not too relevant to this effort.

7 Conclusions

Parallelizing the execution of smart contracts in geth seems feasible. The main challenges are in ensuring that the synchronization overhead does not overshadow the performance benefit that comes from parallel execution. In this document we have explored these issues and outlined courses of action towards acheiving this goal, and identified potential challenges and pitfalls.

Acknowledgments: This work is partially funded by a grant from the Ethereum Foundation, as part of the 2025 Academic Grants Round.

References

[1] Mohammad Javad Amiri, Divyakant Agrawal, and Amr El Abbadi. ParBlockchain: Leveraging Transaction Parallelism in Permissioned Blockchain Systems. In *Proc. of the 39th IEEE International Conference on Distributed Computing Systems (ICDCS)*, pages 1337–1347, 2019.

[2] Dvir David Biton, Roy Friedman, and Yaron Hay. Ethereum Conflicts Graphed. arXiv 2507.20196, 2025. `https://arxiv.org/abs/2507.20196`.

[3] Daniel Brélaz. New Methods to Color the Vertices of a Graph. *Communications of ACM*, 22(4):251—-256, April 1979.

[4] Thomas Dickerson, Paul Gazzillo, Maurice Herlihy, and Eric Koskinen. Adding Concurrency to Smart Contracts. In *Proc. of the ACM Symposium on Principles of Distributed Computing (PODC)*, pages 303–312, 2017.

[5] Erigon. Erigon EVM, 2025. `https://erigon.tech/` and `https://github.com/erigontech/erigon`.

[6] Ethereum Foundation. Go Ethereum. `https://github.com/ethereum/go-ethereum/`, accssed on 06/08/2025, Sprouted Seed Vial (v1.16.2), 2025.

[7] Ethereum Foundation. Introduction to smart contracts, 2025. `https://ethereum.org/en/developers/docs/smart-contracts/`.

[8] Lioba Heimbach, Quentin Kniep, Yann Vonlanthen, Roger Wattenhofer, and Patrick Züst. Dissecting the EIP-2930 Optional Access Lists. arXiv 312.06574, 2023.

[9] Neon. Leveraging parallel transactions on Neon EVM, 2025. `https://www.neonevm.org/blog/leveraging-parallel-transactions-on-neon-evm`.

[10] Nevermind. Nethermind Ethereum Client, 2025. `https://github.com/NethermindEth/nethermind`.

[11] Vikram Saraph and Maurice Herlihy. An Empirical Study of Speculative Concurrency in Ethereum Smart Contracts. Proc. of Tokenomics, 2019.

[12] Vikram Saraph and Maurice Herlihy. An Empirical Study of Speculative Concurrency in Ethereum Smart Contracts. In *International Conference on Blockchain Economics, Security and Protocols (Tokenomics)*, volume 71. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2020.